

UNIVERSIDADE FEDERAL DO MARANHÃO

Curso de Ciência da Computação • Processamento de Imagem

PLANO DE EXECUÇÃO DO PROJETO

Detecção e Quantificação de Doenças em Folhas de Plantas

utilizando Técnicas Clássicas de Processamento de Imagem

Início:	05 de maio de 2026
Entrega:	06 de julho de 2026
Duração:	9 semanas (63 dias)
Tecnologia:	Python 3.10+ OpenCV 4.x NumPy Matplotlib
Dataset:	PlantVillage — tomate + batata (3+ classes)

1. Ideia Refinada e Escopo Técnico

O projeto consiste em construir um pipeline computacional inteiramente em Python + OpenCV para detectar e quantificar doenças em folhas de plantas, usando exclusivamente técnicas clássicas de processamento de imagem — sem redes neurais ou aprendizado profundo.

1.1 Objetivo e Definição de Sucesso

- Entrada: imagem RGB de folha de planta (dataset PlantVillage, 256×256 px, fundo uniforme).
- Saída: porcentagem de área afetada + nível de severidade (Saudável / Leve / Moderada / Grave) + painel visual de 4 subplots.
- Critério de sucesso: pipeline produz classificação de severidade visualmente coerente com os rótulos do PlantVillage em ≥80% das imagens de teste.

1.2 Escopo Técnico — O que ESTÁ incluído

- 4 estágios sequenciais: pré-processamento → segmentação da folha → detecção de lesões → análise quantitativa.
- Segmentação por threshold HSV manual (`cv2.inRange`), com morfologia matemática para refinamento.
- Histograma do canal H para inspeção das distribuições de cor (saudável vs. doente).
- Classificação por limiar fixo: Saudável <5%, Leve 5–20%, Moderada 20–50%, Grave >50%.
- Validação visual com subconjunto de ~150–300 imagens (tomate + batata, 3+ classes).
- Relatório final com análise de limitações e discussão dos resultados.

1.3 O que NÃO está incluído (fora do escopo)

- Redes neurais, CNNs, transfer learning.
- Identificação da espécie de doença (apenas presença e severidade).
- Imagens em campo aberto ou com fundo não uniforme.
- Treinamento / otimização automática de parâmetros.

Suposições explícitas do planejamento:

(1) Você trabalhará sozinho. (2) Dedicção média de ~10–12h/semana. (3) Ambiente local ou Colab. (4) Leitura dos artigos na Semana 1, antes de qualquer implementação. (5) Arquitetura de módulos definida antes de escrever código.

2. Arquitetura do Projeto

Antes de escrever qualquer código, a arquitetura do projeto deve ser definida explicitamente (tarefa F0.7). O diagrama abaixo descreve a estrutura recomendada de pastas, módulos Python e notebooks Jupyter.

2.1 Estrutura de Pastas Recomendada

A estrutura separa claramente o código de produção (módulos .py), os ambientes interativos de exploração (notebooks), os dados (imagens) e os resultados (CSVs, imagens output).

```
plant_disease/
├── data/
│   ├── selected/           # subconjunto curado do PlantVillage
│   │   ├── tomato_healthy/
│   │   ├── tomato_blight/
│   │   └── potato_early_blight/
│   └── ...
├── utils/
│   ├── io.py               # load_image(), load_batch(), save_result()
│   ├── color.py            # to_rgb(), to_hsv(), to_gray()
│   ├── stage1_preproc.py    # preprocess(img) → (rgb, hsv, blurred)
│   ├── stage2_leaf_seg.py   # segment_leaf(hsv) → (mask, area_px)
│   ├── stage3_lesion.py     # detect_lesions(hsv, leaf_mask) → (mask, contours)
│   ├── stage4_analysis.py   # analyze(l_mask, d_mask) → (pct, severity, hist)
│   └── pipeline.py          # process_image(path) → dict completo
├── notebooks/
│   ├── 01_preproc.ipynb
│   ├── 02_leaf_seg.ipynb
│   ├── 03_lesion.ipynb
│   └── 04_pipeline.ipynb    # batch + tabela de resultados
├── results/
│   ├── csv_results.csv      # image, leaf_px, lesion_px, pct, severity
│   └── requirements.txt
└── ...
```

2.2 Interfaces entre Módulos

Cada estágio deve ter uma interface de entrada e saída bem definida antes de ser implementado. Isso garante que a integração (F4.6) seja direta.

Módulo / Arquivo	Responsabilidade	Entrada → Saída	Funções Principais
utils/io.py	Carregamento e salvamento de imagens	path → np.ndarray (BGR)	load_image(), load_batch(), save_result()
utils/color.py	Conversões de espaço de cor	BGR → RGB / HSV / GRAY	to_rgb(), to_hsv(), to_gray()
stage1_preproc.py	Pré-processamento completo	BGR → BGR normalizado + HSV	preprocess(img, size=256) → (rgb, hsv, blurred)
stage2_leaf_seg.py	Segmentação da folha	HSV → máscara binária da folha	segment_leaf(hsv) → (leaf_mask, leaf_area_px)
stage3_lesion.py	Detecção de lesões	HSV + leaf_mask → lesion_mask	detect_lesions(hsv, leaf_mask) → (lesion_mask, contours)
stage4_analysis.py	Análise quantitativa e classificação	máscaras → métricas + viz	analyze(leaf_mask, lesion_mask) → (pct, severity, hist)

Módulo / Arquivo	Responsabilidade	Entrada → Saída	Funções Principais
pipeline.py	Integração end-to-end	path → resultado completo	<code>process_image(path)</code> → dict com todas as métricas
notebooks/01_preproc.ipynb	Exploração pré-processamento	—	Visualização interativa dos estágios 1
notebooks/02_leaf_seg.ipynb	Exploração segmentação	—	Tuning visual dos ranges HSV
notebooks/03_lesion.ipynb	Exploração detecção lesões	—	Tuning visual das máscaras de doença
notebooks/04_pipeline.ipynb	Pipeline completo + batch	—	Execução em batch, tabela de resultados
data/selected/	Subconjunto de imagens	—	~150–300 imgs / 3 classes / 2 espécies
results/csv_results.csv	Tabela de resultados batch	—	image, leaf_px, lesion_px, pct, severity

2.3 Convenções de Desenvolvimento

- Tipos NumPy: imagens BGR e RGB em `np.uint8`. Máscaras binárias em `np.uint8` (0 ou 255). HSV em `np.uint8`.
- Cada módulo de estágio deve ser testável isoladamente (sem depender de outros módulos).
- Cada notebook deve importar os módulos `.py` correspondentes — não duplicar código.
- Commits frequentes no Git com mensagens descritivas por subtarefa concluída.

3. Estrutura WBS — Work Breakdown Structure

O projeto está dividido em 6 fases (F0–F6), com 44 subtarefas. Cada subtarefa é executável isoladamente e tem critério de conclusão definível.

ID	Subtarefa	Fase	Tipo	Complexidade
F0.1	Criar e configurar ambiente virtual Python (venv)	Setup	setup	Baixa
F0.2	Instalar dependências: opencv-python, numpy, matplotlib, jupyter	Setup	setup	Baixa
F0.3	Download e organização do dataset PlantVillage (tomate + batata)	Setup	setup	Baixa
F0.4	Selecionar subconjunto de imagens (50–100 img/classe, 3+ classes)	Setup	setup	Baixa
F0.5	Leitura do artigo Mohanty et al. 2016 (PlantVillage)	Setup	estudo	Baixa
F0.6	Leitura do artigo Singh & Misra 2021 (pipeline clássico HSV+GLCM)	Setup	estudo	Baixa
F0.7	Definir arquitetura: estrutura de pastas, módulos e interfaces	Setup	arquitetura	Média
F0.8	Criar esqueleto do repositório e notebooks base	Setup	setup	Baixa
F1.1	Implementar carregamento de imagens com cv2.imread() (single + batch)	Pré-proc.	implementação	Baixa
F1.2	Implementar conversão BGR→RGB para visualização (matplotlib)	Pré-proc.	implementação	Baixa
F1.3	Implementar conversão BGR→HSV (espaço principal de análise)	Pré-proc.	implementação	Baixa
F1.4	Implementar suavização com cv2.GaussianBlur() (kernel 5×5)	Pré-proc.	implementação	Baixa
F1.5	Implementar redimensionamento para 256×256 (cv2.resize)	Pré-proc.	implementação	Baixa
F1.6	Criar notebook de pré-processamento com visualizações e comentários	Pré-proc.	validação	Baixa
F2.1	Implementar segmentação HSV dos pixels verdes (cv2.inRange)	Seg. Folha	implementação	Média
F2.2	Implementar limiarização de Otsu como método alternativo	Seg. Folha	implementação	Média
F2.3	Implementar operações morfológicas: abertura e fechamento da máscara	Seg. Folha	implementação	Média
F2.4	Implementar extração do maior contorno (cv2.findContours + sort por área)	Seg. Folha	implementação	Média
F2.5	Implementar aplicação de máscara (cv2.bitwise_and)	Seg. Folha	implementação	Baixa
F2.6	Ajuste fino dos ranges HSV via inspeção visual interativa	Seg. Folha	ajuste	Alta

ID	Subtarefa	Fase	Tipo	Complexidade
F2.7	Validação visual com 20+ imagens (saudáveis + doentes, tomate + batata)	Seg. Folha	validação	Média
F3.1	Definir ranges HSV para lesões: amarelos (H 15–35) e marrons (H 0–15, 160–179)	Det. Lesões	implementação	Alta
F3.2	Implementar segmentação de regiões doentes (cv2.inRange multi-range)	Det. Lesões	implementação	Alta
F3.3	Implementar interseção lesão × máscara folha (cv2.bitwise_and)	Det. Lesões	implementação	Média
F3.4	Implementar refinamento morfológico da máscara de lesão	Det. Lesões	implementação	Média
F3.5	Implementar detecção de contornos individuais de lesão	Det. Lesões	implementação	Média
F3.6	Implementar visualização: contornos sobre imagem original	Det. Lesões	implementação	Baixa
F4.1	Implementar contagem de pixels (np.sum da máscara binária)	Análise Quant.	implementação	Baixa
F4.2	Implementar cálculo de porcentagem afetada (lesão / folha × 100)	Análise Quant.	implementação	Baixa
F4.3	Implementar classificador de severidade por limiar fixo (4 níveis)	Análise Quant.	implementação	Baixa
F4.4	Implementar histograma do canal H (cv2.calcHist + normalização)	Análise Quant.	implementação	Média
F4.5	Implementar painel de visualização final (4 subplots com matplotlib)	Análise Quant.	implementação	Baixa
F4.6	Integrar os 4 estágios em pipeline end-to-end (função process_image)	Análise Quant.	integração	Alta
F5.1	Executar pipeline em batch com 50+ imagens (saudáveis + doentes)	Validação	validação	Média
F5.2	Analisar casos de falha: falsos positivos e negativos	Validação	ajuste	Alta
F5.3	Ajuste fino de parâmetros baseado nos erros observados	Validação	ajuste	Alta
F5.4	Gerar tabela de resultados consolidada (por imagem)	Validação	validação	Baixa
F5.5	Documentar limitações e casos extremos observados	Validação	validação	Baixa
F6.1	Escrever seção de metodologia do relatório	Relatório	documentação	Baixa
F6.2	Inserir visualizações e tabela de resultados no relatório	Relatório	documentação	Baixa
F6.3	Escrever análise e discussão dos resultados	Relatório	documentação	Média
F6.4	Limpeza final dos notebooks (comentários, organização)	Relatório	documentação	Baixa

ID	Subtarefa	Fase	Tipo	Complexidade
F6.5	Revisão final do relatório e notebooks	Relatório	validação	Baixa
F6.6	Entrega final (06/07/2026)	Relatório	entrega	Baixa

Legenda de Complexidade

- Baixa — tarefa direta, bem documentada no OpenCV, $\leq 1h$.
- Média — requer experimentação ou múltiplos passos, 1–3h.
- Alta — iterativa, com ajuste de parâmetros ou integração entre componentes, 3–6h+.

4. Estimativas de Tempo (PERT)

Estimativas calculadas com a fórmula PERT: Esperado = (mín + 4×provável + máx) / 6. Valores em horas.

ID	Subtarefa	Mín (h)	Máx (h)	Esperado (h)	Complexidade	Tipo
F0.1	Configurar ambiente virtual Python	0.5	1.0	0.6	Baixa	setup
F0.2	Instalar dependências (pip + jupyter)	0.3	0.5	0.3	Baixa	setup
F0.3	Download e organização do dataset	1.0	3.0	2.0	Baixa	setup
F0.4	Selecionar subconjunto de imagens	0.5	1.5	1.0	Baixa	setup
F0.5	Leitura Mohanty et al. 2016	1.5	3.0	2.1	Baixa	estudo
F0.6	Leitura Singh & Misra 2021	1.5	3.0	2.1	Baixa	estudo
F0.7	Definir arquitetura e estrutura de pastas	1.0	3.0	2.0	Média	arquitetura
F0.8	Criar esqueleto do repositório e notebooks	0.5	1.5	1.0	Baixa	setup
F1.1	Carregar imagens (single + batch)	0.5	1.0	0.7	Baixa	implementação
F1.2	Conversão BGR→RGB	0.3	0.5	0.3	Baixa	implementação
F1.3	Conversão BGR→HSV	0.3	0.5	0.3	Baixa	implementação
F1.4	GaussianBlur (suavização)	0.3	1.0	0.5	Baixa	implementação
F1.5	Redimensionamento 256×256	0.3	0.5	0.3	Baixa	implementação
F1.6	Notebook pré-processamento + visualizações	1.0	2.0	1.5	Baixa	validação
F2.1	Segmentação HSV pixels verdes	1.0	3.0	2.0	Média	implementação
F2.2	Limiarização de Otsu (alternativo)	0.5	1.5	1.0	Média	implementação
F2.3	Morfologia: abertura + fechamento da máscara	1.0	2.5	1.6	Média	implementação
F2.4	Extração do maior contorno	1.0	2.5	1.6	Média	implementação
F2.5	Aplicação de máscara bitwise_and	0.3	0.5	0.3	Baixa	implementação
F2.6	Ajuste fino ranges HSV (tuning)	2.0	5.0	3.2	Alta	ajuste
F2.7	Validação visual 20+ imagens	1.5	3.0	2.1	Média	validação
F3.1	Definir ranges HSV para lesões	1.5	4.0	2.6	Alta	implementação
F3.2	Segmentação doença (multi-range inRange)	1.0	3.0	2.0	Alta	implementação
F3.3	Interseção lesão × máscara folha	0.5	1.0	0.7	Média	implementação
F3.4	Morfologia da máscara de lesão	1.0	2.0	1.5	Média	implementação
F3.5	Detecção de contornos individuais	1.0	2.0	1.5	Média	implementação
F3.6	Visualização: contornos sobre imagem	0.5	1.0	0.7	Baixa	implementação
F4.1	Contagem de pixels (np.sum)	0.3	0.5	0.3	Baixa	implementação

ID	Subtarefa	Mín (h)	Máx (h)	Esperado (h)	Complexidade	Tipo
F4.2	Cálculo porcentagem afetada	0.3	0.5	0.3	Baixa	implementação
F4.3	Classificador de severidade (4 níveis)	0.5	1.0	0.7	Baixa	implementação
F4.4	Histograma canal H (cv2.calcHist)	1.0	2.0	1.5	Média	implementação
F4.5	Painel 4 subplots (matplotlib)	1.0	2.5	1.6	Baixa	implementação
F4.6	Integração pipeline end-to-end	2.0	5.0	3.2	Alta	integração
F5.1	Batch 50+ imagens	2.0	4.0	3.0	Média	validação
F5.2	Análise de falhas	2.0	4.0	3.0	Alta	ajuste
F5.3	Ajuste fino pós-análise	2.0	6.0	3.7	Alta	ajuste
F5.4	Tabela de resultados consolidada	1.0	2.0	1.5	Baixa	validação
F5.5	Documentar limitações	1.0	2.0	1.5	Baixa	validação
F6.1	Seção metodologia (relatório)	2.0	4.0	3.0	Baixa	documentação
F6.2	Inserir figuras e tabelas	1.0	2.0	1.5	Baixa	documentação
F6.3	Análise e discussão	2.0	4.0	3.0	Média	documentação
F6.4	Limpeza notebooks	1.0	2.0	1.5	Baixa	documentação
F6.5	Revisão final	1.0	3.0	2.0	Baixa	validação
F6.6	Entrega final 06/07/2026	0.5	1.0	0.6	Baixa	entrega

Totais estimados:

Total mínimo: ~40h | Total esperado: ~57h | Total máximo: ~84h | Com buffer 15%: ~65h (esperado) ≈ 7–8h por semana.

5. Dependências e Ordem de Execução

5.1 Tabela de Dependências

ID	Subtarefa	Depende de	Observação
F0.7	Definir arquitetura	F0.5, F0.6	Ler os artigos antes de desenhar a arquitetura
F0.8	Criar esqueleto repositório	F0.7	Arquitetura deve estar definida
F1.1	Carregar imagens (batch)	F0.3, F0.4, F0.8	Dataset e estrutura de pastas devem existir
F1.2	Conversão BGR→RGB	F1.1	Depende do carregamento
F1.3	Conversão BGR→HSV	F1.1	Depende do carregamento
F1.4	GaussianBlur	F1.1	Pode rodar em paralelo com F1.2, F1.3
F1.5	Resize 256×256	F1.1	Pode rodar em paralelo com F1.2–F1.4
F1.6	Notebook pré-processamento	F1.1–F1.5	Todas as funções devem estar implementadas
F2.1	Segmentação HSV verde	F1.3, F1.6	Depende da imagem em HSV normalizada
F2.2	Otsu (alternativo)	F1.1, F1.6	Pode ser desenvolvido em paralelo com F2.1
F2.3	Morfologia máscara folha	F2.1	Precisa da máscara inicial
F2.4	Extração maior contorno	F2.3	Precisa da máscara morfológica
F2.5	bitwise_and aplicação	F2.4	Precisa do contorno/máscara final
F2.6	Ajuste fino HSV	F2.1–F2.5	Tuning iterativo — caminho crítico
F2.7	Validação visual folha	F2.6	Só validar após ajuste fino
F3.1	Definir ranges lesões HSV	F2.7	Conhecimento de F2 necessário; caminho crítico
F3.2	Segmentação doença	F3.1	Depende dos ranges definidos
F3.3	Interseção lesão × folha	F3.2, F2.5	Precisa da máscara de folha e de doença
F3.4	Morfologia lesão	F3.3	Refina a interseção
F3.5	Contornos individuais lesão	F3.4	Precisa da máscara refinada
F3.6	Viz contornos	F3.5	Último passo visual do estágio 3
F4.1	Contagem pixels	F2.4, F3.4	Precisa das duas máscaras finais
F4.2	Porcentagem afetada	F4.1	Depende da contagem
F4.3	Classificador severidade	F4.2	Depende da porcentagem
F4.4	Histograma canal H	F3.4, F2.5	Precisa das regiões segmentadas
F4.5	Painel visualização	F4.4, F4.3	Último passo visual
F4.6	Pipeline end-to-end	F4.1–F4.5	Integração: caminho crítico
F5.1	Batch 50+ imagens	F4.6	Pipeline deve estar funcional

ID	Subtarefa	Depende de	Observação
F5.2	Análise falhas	F5.1	Depende dos resultados do batch
F5.3	Ajuste fino pós-análise	F5.2	Pode iterar com F5.1
F5.4	Tabela resultados	F5.1	Pode rodar em paralelo com F5.2, F5.3
F5.5	Documentar limitações	F5.2	Após análise de falhas
F6.1	Metodologia relatório	F4.6	Pode iniciar assim que pipeline estiver pronto
F6.2	Figuras e tabelas	F5.4	Depende dos resultados finais
F6.3	Análise e discussão	F5.2, F5.5	Depende da análise de falhas
F6.4	Limpeza notebooks	F5.3	Após ajuste fino final
F6.5	Revisão final	F6.1–F6.4	Tudo deve estar pronto
F6.6	Entrega	F6.5	Último passo — 06/07/2026

5.2 Caminho Crítico

As tarefas que **NÃO** podem atrasar sem impactar o prazo final são:

⚡ Caminho Crítico:

F0.5/F0.6 (leitura artigos) → F0.7 (arquitetura) → F2.1 (seg. HSV) → F2.6 (ajuste HSV) ★ → F3.1 (ranges lesão) ★ → F3.2 (seg. doença) → F4.6 (integração pipeline) ★ → F5.1/F5.2 (validação + análise falhas) ★ → F6.3 (discussão) → F6.5 (revisão) → Entrega. ★ = tarefas com maior risco de atraso.

5.3 Tarefas Paralelizáveis

- F1.2, F1.3, F1.4, F1.5 podem ser implementadas em paralelo (todas dependem apenas de F1.1).
- F2.2 (Otsu) pode ser desenvolvida em paralelo com F2.1 (HSV) como alternativa.
- F5.4 (tabela de resultados) pode rodar em paralelo com F5.2/F5.3 (análise de falhas).
- F6.1 (metodologia) pode começar na S7, junto com F5.x.

6. Cronograma Detalhado

6.1 Visão Semanal

Semana	Período	Atividades Principais	Entregável	Status
S1	05/05–11/05	Leitura artigos (Mohanty + Singh) Download PlantVillage Config. ambiente Definir arquitetura e estrutura de pastas (F0.1–F0.8)	Ambiente pronto + dataset local + arquitetura doc.	Normal
S2	12/05–18/05	Estágio 1: carregamento, conversões BGR→RGB/HSV, GaussianBlur, resize 256×256 Notebook pré-processamento (F1.1–F1.6)	Notebook: Pré-processamento	Normal
S3	19/05–25/05	Estágio 2: segmentação HSV verde, Otsu, morfologia, extração maior contorno, bitwise_and (F2.1–F2.5)	Máscara folha (versão inicial)	Crítico
S4	26/05–01/06	Ajuste fino ranges HSV Validação visual 20+ imagens Refinar morfologia conforme erros observados (F2.6–F2.7)	Máscara de folha validada ≥80% imgs	🚩 Breakpoint 2
S5	02/06–08/06	Estágio 3: definir ranges lesão segmentação doença interseção morfologia contornos viz (F3.1–F3.6)	Notebook: Detecção de Lesões	Crítico
S6	09/06–15/06	Estágio 4: pixel count, %, classificador, histograma, painel viz Integração pipeline end-to-end (F4.1–F4.6)	Pipeline completo e integrado	🚩 Breakpoint 4
S7	16/06–22/06	Batch 50+ imgs Análise de falsos positivos/negativos Ajuste fino final Tabela de resultados (F5.1–F5.5)	Resultados validados + tabela	🚩 Breakpoint 5
S8	23/06–29/06	Escrever metodologia Inserir figuras/tabelas Análise e discussão Limpeza notebooks (F6.1–F6.4)	Relatório (rascunho)	Normal
S9	30/06–06/07	Revisão final relatório e notebooks Entrega (F6.5–F6.6)	🎓 ENTREGA — 06/07/2026	Crítico

6.2 Breakpoints do Projeto

Breakpoints são marcos de validação obrigatória. Antes de avançar, o critério de saída do breakpoint deve ser satisfeito. Se não for, dedique tempo adicional antes de prosseguir para a próxima fase.

🚩 BREAKPOINT 1 — Fundação Pronta (fim S1 — 11/05)

Critério de saída: Ambiente Python funcionando, dataset baixado e organizado em subpastas, artigos Mohanty e Singh lidos, documento de arquitetura (F0.7) escrito com estrutura de pastas + interfaces dos módulos.

🚩 BREAKPOINT 2 — Pré-processamento Validado (fim S2 — 18/05)

Critério de saída: Notebook 01_preproc.ipynb carrega, converte (BGR→RGB/HSV), suaviza e redimensiona imagens de forma reprodutível. Visualizações corretas para pelo menos 5 imagens de teste.

**BREAKPOINT 3 — Máscara de Folha Funcionando (fim S4 — 01/06)**

Critério de saída: Segmentação produz máscara binária da folha com $\geq 80\%$ de acerto visual em 20+ imagens. Buracos internos fechados por morfologia. Ruídos de fundo eliminados. Parâmetros HSV documentados.

**BREAKPOINT 4 — Pipeline Completo Integrado (fim S6 — 15/06)**

Critério de saída: Função `process_image(path)` retorna: `leaf_mask`, `lesion_mask`, `pct_affected`, `severity`, painel `matplotlib`. Testado com pelo menos 10 imagens (saudáveis + doentes). Integração sem erros de interface.

**BREAKPOINT 5 — Validação Concluída (fim S7 — 22/06)**

Critério de saída: Batch de 50+ imagens executado. Tabela CSV de resultados gerada. Casos de falha identificados e documentados. Ajuste fino aplicado. Limitações registradas.

**BREAKPOINT 6 — Pronto para Entrega (fim S8 — 29/06)**

Critério de saída: Relatório rascunho completo (metodologia + resultados + discussão). Notebooks limpos e comentados. Nenhuma tarefa crítica pendente. Margem de 1 semana para revisão.

6.3 Resumo de Duração

 Duração Mínima	 Duração Realista	 Com Buffer (15%)
~40 horas totais(~4,5h/semana)	~57 horas totais(~6,3h/semana)	~65 horas totais(~7,2h/semana)

7. Riscos e Mitigação

7.1 Riscos Técnicos

Risco	Probabilidade	Impacto	Estratégia de Mitigação
[T1] Ranges HSV para lesões não generalizam para múltiplas doenças	Alta	Alto	Testar com classes distintas desde a S4. Usar union de ranges. Documentar parâmetros por doença.
[T2] Segmentação da folha falha com fundo não uniforme	Média	Médio	Dataset PlantVillage tem fundo uniforme — baixo risco no escopo. Documentar como limitação.
[T3] Morfologia não fecha buracos grandes na máscara	Média	Médio	Testar kernels maiores (11×11, 15×15). Combinar MORPH_CLOSE + MORPH_DILATE.
[T4] Falsos positivos: pixels amarelos/marrons saudáveis detectados como lesão	Alta	Alto	Intersectar sempre com máscara de folha. Refinar lower bound do canal H. Testar threshold mínimo de contorno.
[T5] Pipeline integrado (F4.6) com bugs de interface entre módulos	Média	Médio	Definir interfaces claras na fase F0.7 (arquitetura). Testar módulos isolados antes da integração.

7.2 Riscos de Execução

Risco	Probabilidade	Impacto	Estratégia de Mitigação
[E1] Tuning HSV (F2.6) consumir mais tempo que o previsto	Alta	Alto	Reservar 5 horas para este passo. Se travar após 3h, avançar com threshold aproximado e refinar na S7.
[E2] Semana 3 (segmentação) sobrecarregada — tarefa mais complexa	Média	Alto	Distribuir F2.1–F2.5 para S3 e mover ajuste fino (F2.6–F2.7) para S4 conforme cronograma.
[E3] Subestimar tempo de análise de falhas (F5.2–F5.3)	Média	Médio	Separar sessões de análise das sessões de ajuste. Usar imagens-âncora para comparação.

7.3 Riscos Externos

Risco	Probabilidade	Impacto	Estratégia de Mitigação
[X1] Download do dataset PlantVillage lento ou indisponível (GitHub LFS)	Média	Médio	Utilizar Kaggle mirror: kaggle.com/datasets/emmarex/plantdisease . Baixar na S1.
[X2] Ambiente de desenvolvimento instável (Colab desconectando)	Baixa	Baixo	Preferir ambiente local. Salvar checkpoints frequentes. Versionar notebooks no Git.

8. Guia Prático de Execução

8.1 Primeiros Passos — Faça isso antes de qualquer código

1. Criar o ambiente virtual: `python -m venv venv && source venv/bin/activate` (ou `venv\Scripts\activate` no Windows).
2. Instalar dependências: `pip install opencv-python numpy matplotlib jupyter`.
3. Baixar o dataset PlantVillage: github.com/spMohanty/PlantVillage-Dataset. Alternativa: Kaggle mirror.
4. Criar a estrutura de pastas exatamente como mostrado na seção 2.1.
5. Ler Mohanty et al. (2016) com foco na seção de segmentação por espaço de cor (seção 2).
6. Ler Singh & Misra (2021) com foco na seção de features HSV e pipeline (seções 3–4).
7. Escrever o documento de arquitetura (F0.7) — 1 página com interfaces dos módulos.

8.2 O que validar em cada estágio — Critérios de "pronto"

Estágio 1 — Pré-processamento

- `cv2.imread()` retorna `ndarray (H, W, 3) dtype uint8?` ✓
- HSV: canal H na faixa `[0, 179]`, S e V em `[0, 255]`? ✓
- Visualização `matplotlib` mostra imagem com cores corretas (não invertida)? ✓
- Imagem após `GaussianBlur` está visivelmente suavizada mas sem perder forma da folha? ✓

Estágio 2 — Segmentação da Folha

- Máscara binária tem valor 255 dentro da folha e 0 no fundo? ✓
- Após morfologia, não há buracos grandes dentro da folha? ✓
- O maior contorno identificado corresponde à folha (não ao fundo)? ✓
- `cv2.bitwise_and` retorna imagem com fundo preto e folha preservada? ✓

Estágio 3 — Detecção de Lesões

- Máscara de lesão está contida dentro da máscara de folha (nenhum pixel fora)? ✓
- Folhas saudáveis retornam máscara de lesão praticamente vazia (`<1%`)? ✓
- Folhas doentes visualmente têm os contornos de lesão desenhados corretamente? ✓

Estágio 4 — Análise

- Folha saudável classifica como "Saudável" (`<5%`)? ✓
- Histograma canal H mostra pico verde nas saudáveis e amarelo/marrom nas doentes? ✓
- `process_image()` retorna dict com todas as chaves esperadas sem erros? ✓

8.3 Erros Comuns a Evitar

- NUNCA esquecer que OpenCV carrega em BGR, não RGB. Sempre converter antes de exibir no `matplotlib`.
- NUNCA aplicar `GaussianBlur` depois da conversão para HSV — aplicar sobre BGR antes.
- NUNCA calcular área de lesão sem intersectar com a máscara de folha (`cv2.bitwise_and` obrigatório).
- CUIDADO com ranges HSV para tons marrons: H próximo de 0 e 179 são contíguos (o canal H é circular). Use dois `inRange` separados e combine com `cv2.bitwise_or`.
- CUIDADO com kernels morfológicos grandes: podem distorcer o contorno da folha. Começar com `5×5` ou `7×7`.
- NUNCA hardcodar caminhos absolutos nos notebooks. Usar `os.path.join()` com caminhos relativos.

8.4 Critérios de Conclusão do Projeto

- Pipeline `process_image(path)` executa sem erros para qualquer imagem do PlantVillage (tomate + batata).
- Painel de 4 subplots gerado para cada imagem analisada.
- Tabela CSV com resultados de 50+ imagens incluída no repositório.
- Notebooks limpos, comentados e executáveis do início ao fim sem erros.
- Relatório final entregue até 06/07/2026 com metodologia, resultados e discussão.

8.5 Ferramentas de Consulta Rápida

Durante a implementação, mantenha estes recursos abertos:

- Documentação `cv2.inRange`: https://docs.opencv.org/4.x/d2/de8/group__core__array.html
- Documentação `cv2.morphologyEx`: https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html
- Tabela de valores HSV para cores comuns (buscar "HSV color chart OpenCV").
- PlantVillage no Kaggle (mirror): <https://www.kaggle.com/datasets/emmarex/plantdisease>
- Artigo Singh & Misra 2021: <https://arxiv.org/abs/2106.10698>